CS340 : Algorithms and Data Structures

Non-Comparison Sorting

Dr. Ali Moltajaei Farid

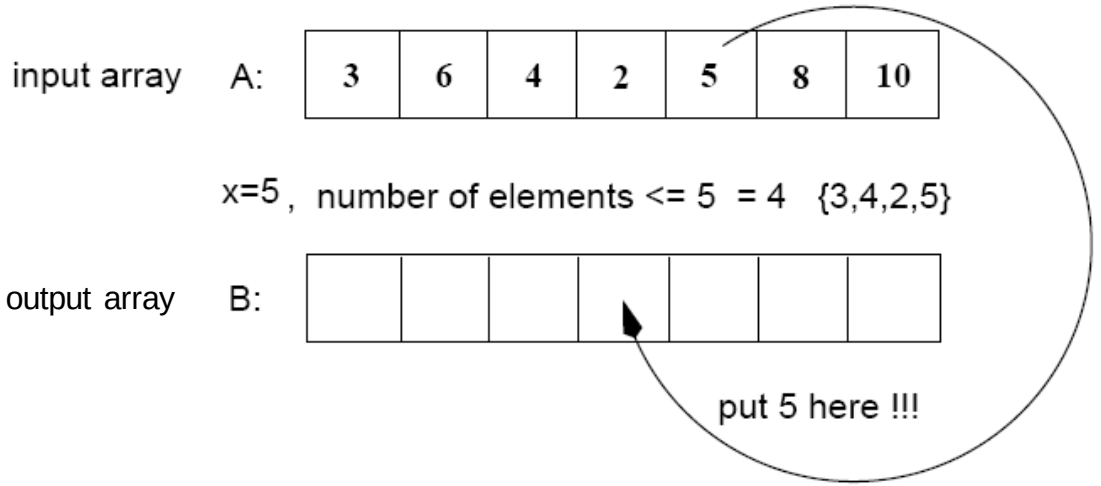
Dept. of Computer Science

Time complexity of Sorting

- Several sorting algorithms have been discussed and the best ones, so far:
  - Heap sort and Merge sort: $O(n \log n)$
  - Quick sort (best one in practice): $O(n \log n)$ on average, $O(n^2)$ worst case
- Can we do better than $O(n \log n)$?
  - No.
  - It can be proven that any comparison-based sorting algorithm will need to carry out at least $O(n \log n)$ operations

Counting Sort

- Assumptions:
  - n integers which are in the range $[0 \ldots r]$
  - r is in the order of n, that is, $r=O(n)$
- Idea:
  - For each element x, find the number of elements $\leq x$
  - Place x into its correct position in the output array



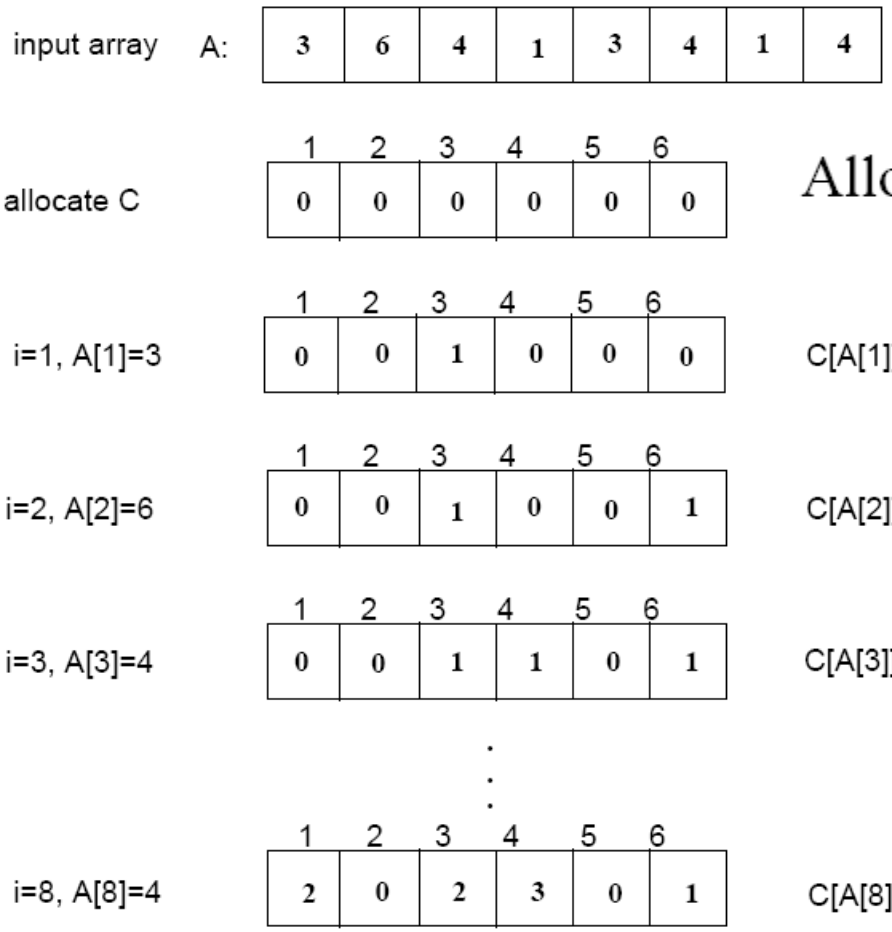
Restrictions on the problem

- Suppose the values in the list to be sorted can repeat but the values have a limit (e.g., values are digits from 0 to 9)
- Sorting, in this case, appears easier
- Is it possible to come up with an algorithm better than O(n log n)?
  - Yes
  - Strategy will not involve comparisons

Step 1

Find the number of times $A[i]$ appears in A

(i.e., frequencies)

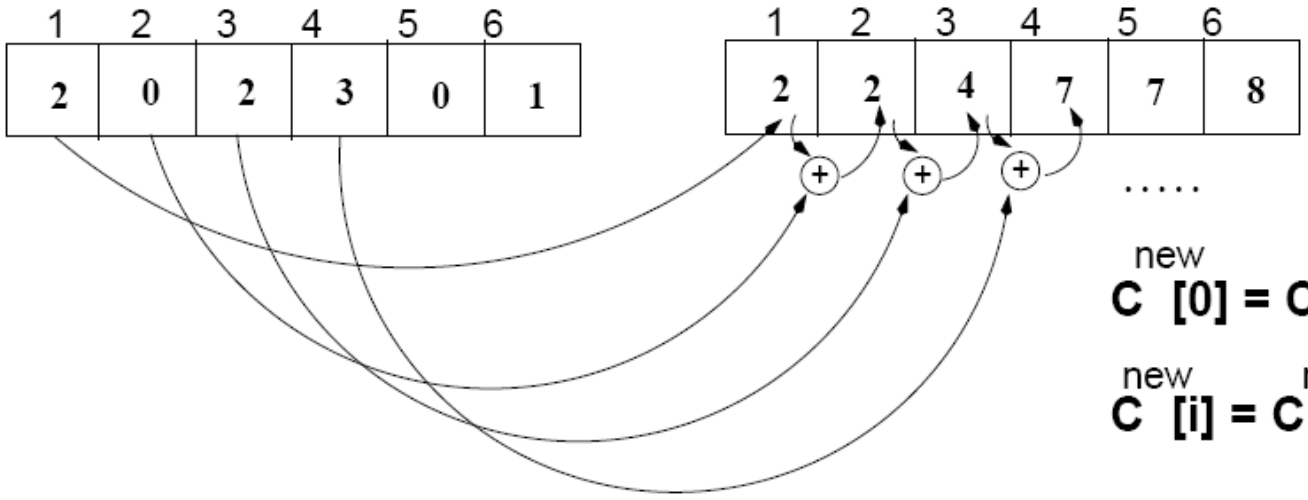


C[i] = number of times element i appears in A

Step 2

Find the number of elements $\leq A[i]$,

C (frequencies) C^{new} (cumulative sums)



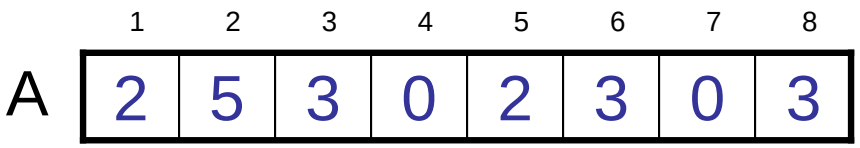
$C^{new}[0] = C^{old}[0]$

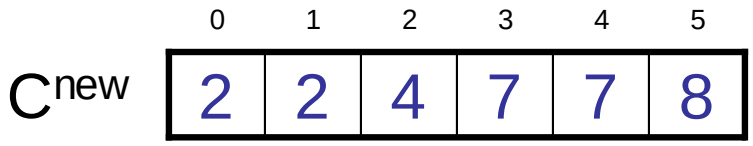
$C^{new}[i] = C^{new}[i-1] + C^{old}[i]$

C[i] = # elements <= i

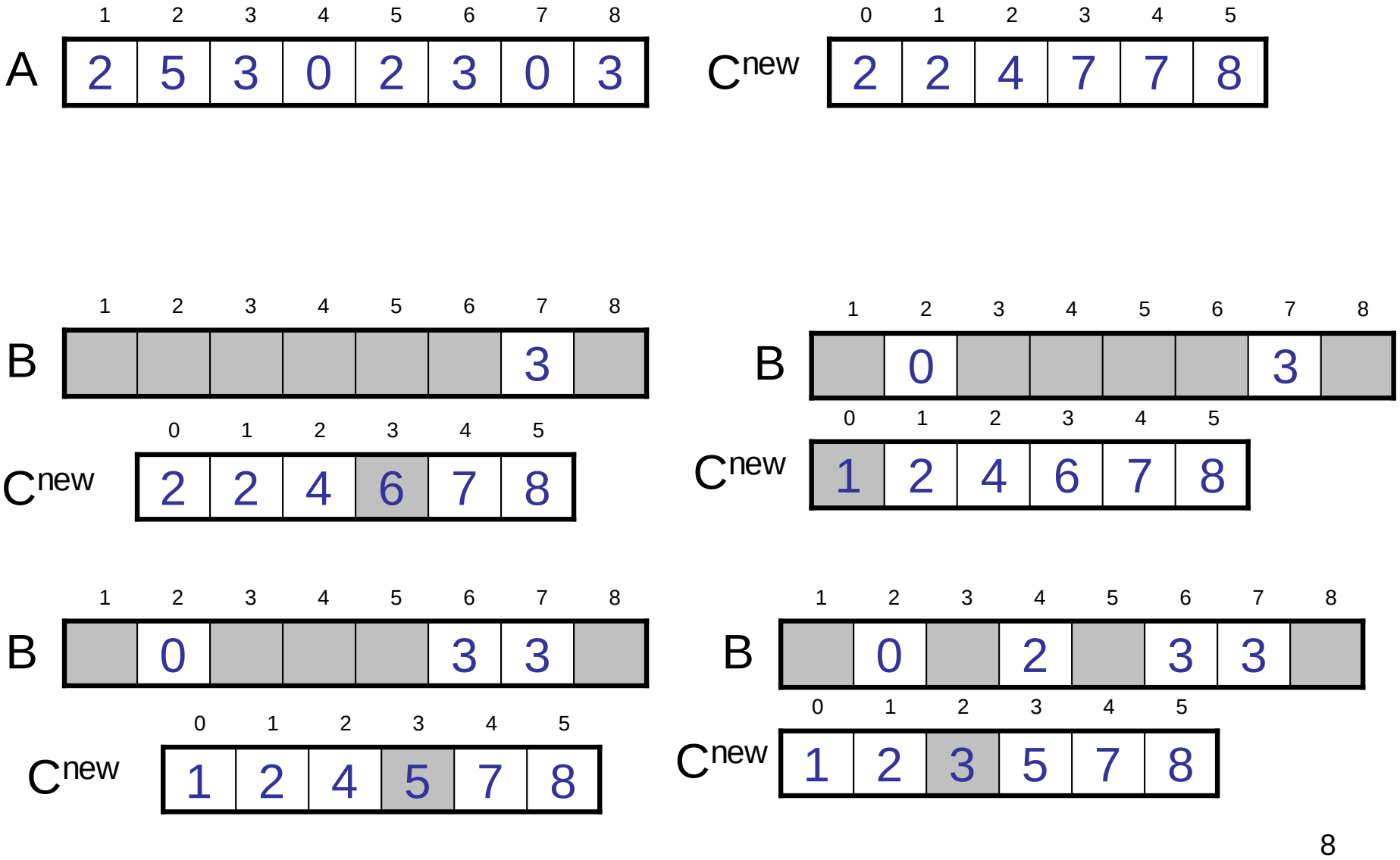
Algorithm

- Start from the last element of A
- Place A[i] at its correct place in the output array
- Decrease C[A[i]] by one

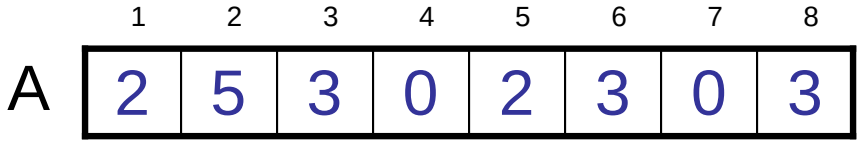


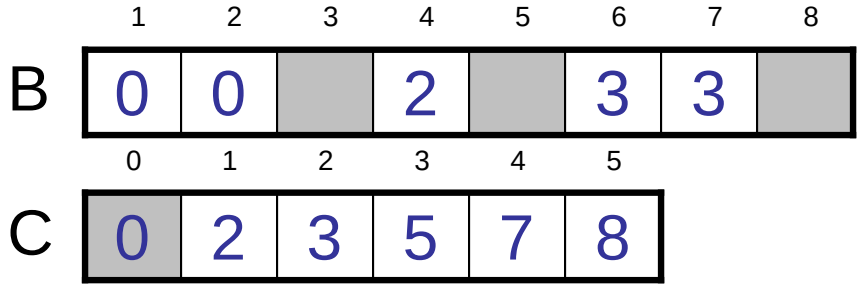


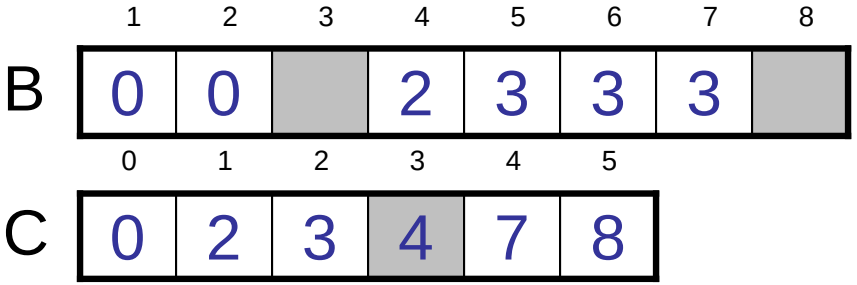
Example

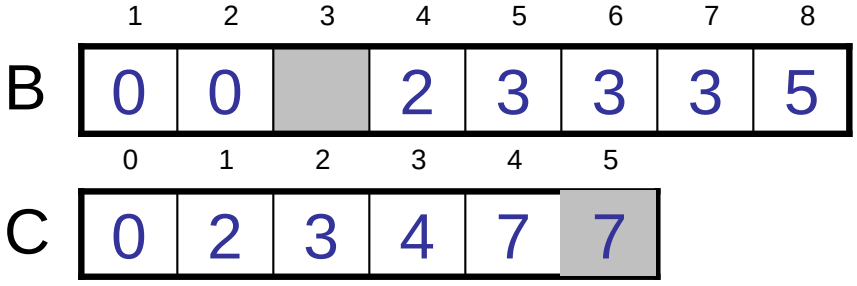

A
1 2 3 4 5 6 7 8
2 5 3 0 2 3 0 3
C new
0 1 2 3 4 5
2 2 4 7 7 8
B
1 2 3 4 5 6 7 8
3
C new
0 1 2 3 4 5
2 2 4 6 7 8
B
1 2 3 4 5 6 7 8
0 3
C new
0 1 2 3 4 5
1 2 4 6 7 8
B
1 2 3 4 5 6 7 8
0 3 3
C new
0 1 2 3 4 5
1 2 4 5 7 8
B
1 2 3 4 5 6 7 8
0 2 3 3
C new
0 1 2 3 4 5
1 2 3 5 7 8

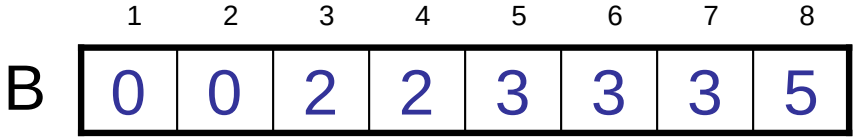
Example (cont.)





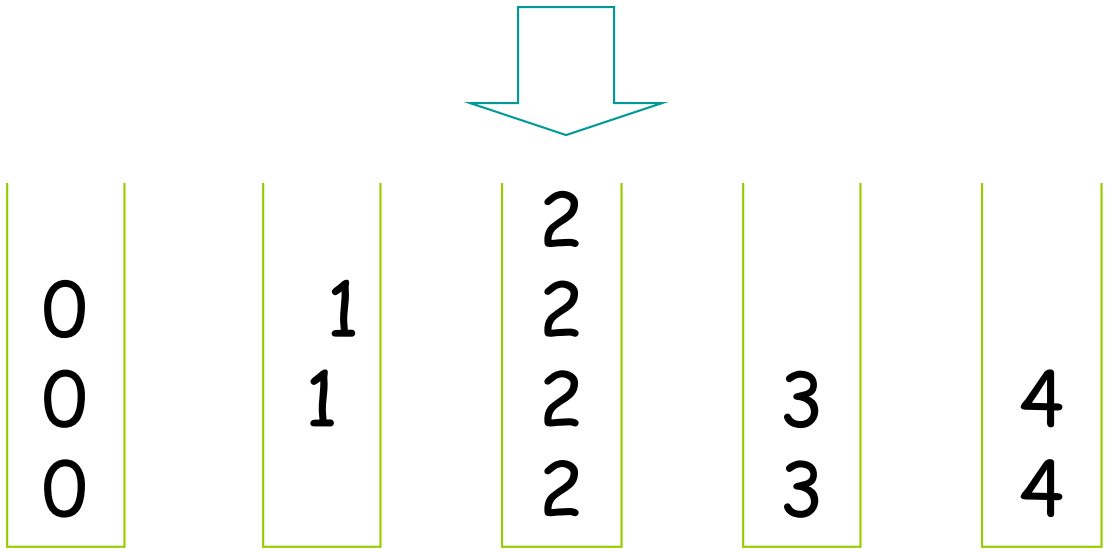






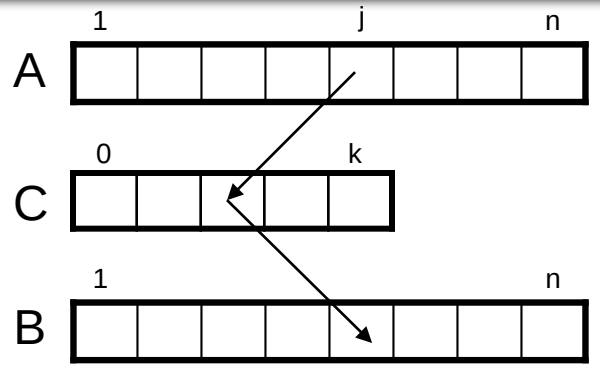
Example (cont.)

4	2	1	2	0	3	2	1	4	0	2	3	0



0	0	0	1	1	2	2	2	2	3	3	4	4

COUNTING-SORT



Alg.: COUNTING-SORT(A, B, n, k)

1. **for** i ← 0 **to** r
2. **do** C[i] ← 0
3. **for** j ← 1 **to** n
4. **do** C[A[j]] ← C[A[j]] + 1
5. ▷ C[i] contains the number of elements equal to i
6. **for** i ← 1 **to** r
7. **do** C[i] ← C[i] + C[i -1]
8. ▷ C[i] contains the number of elements ≤ i
9. **for** j ← n **downto** 1
10. **do** B[C[A[j]]] ← A[j]
11. C[A[j]] ← C[A[j]] - 1

Analysis of Counting Sort

Alg.: COUNTING-SORT(A, B, n, k)

```
1.        for i ← 0 to r                                   } O(r)
2.            do C[ i ] ← 0
3.        for j ← 1 to n                                   } O(n)
4.            do C[A[ j ]] ← C[A[ j ]] + 1
5.        ▷ C[i] contains the number of elements equal to i
6.        for i ← 1 to r                                   } O(r)
7.            do C[ i ] ← C[ i ] + C[i -1]
8.        ▷ C[i] contains the number of elements ≤ i
9.        for j ← n downto 1                               } O(n)
10.           do B[C[A[ j ]]] ← A[ j ]
11.              C[A[ j ]] ← C[A[ j ]] - 1
```

Overall time: $O(n + r)$

Analysis of Counting Sort

- Overall time: $O(n + r)$

- In practice we use COUNTING sort when $r = O(n)$

  $\Rightarrow$ running time is $O(n)$

Radix Sort

- Represents keys as d-digit numbers in some base-k

$$key = x_1 x_2 \ldots x_d \quad \text{where } 0 \le x_i \le k-1$$

- Example: key=15

$$key_{10} = 15,\ d=2,\ k=10 \quad \text{where } 0 \le x_i \le 9$$

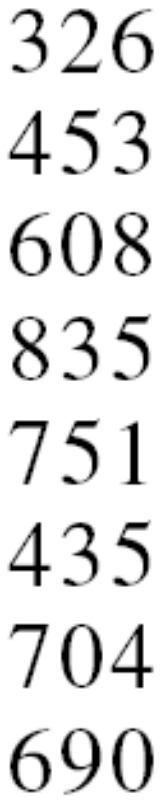
$$key_2 = 1111,\ d=4,\ k=2 \quad \text{where } 0 \le x_i \le 1$$

Radix Sort

- Assumptions

  d=O(1) and k =O(n)

- Sorting looks at one column at a time
  - For a d digit number, sort the least significant digit first
  - Continue sorting on the next least significant digit, until all digits have been sorted
  - Requires only d passes through the list

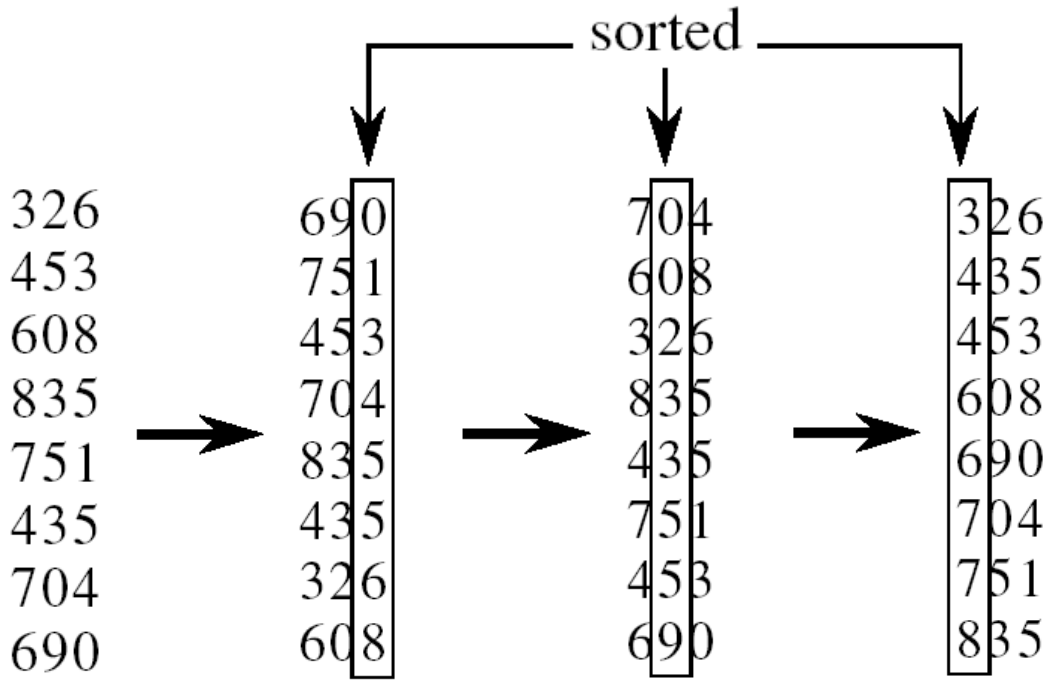


RADIX-SORT

Alg.: RADIX-SORT(*A*, d)

for i ← 1 **to** d

do sort array *A* on digit i



Analysis of Radix Sort

- Given n numbers of d digits each, where each digit may take up to k possible values, RADIX-SORT correctly sorts the numbers in $O(d(n+k))$
  - One pass of sorting per digit takes $O(n+k)$ assuming that we use **counting sort**
  - There are d passes (for each digit)

Analysis of Radix Sort

- Given n numbers of d digits each, where each digit may take up to k possible values, RADIX-SORT correctly sorts the numbers in $O(d(n+k))$
  - Assuming d=O(1) and k=O(n), running time is O(n)

Bucket Sort

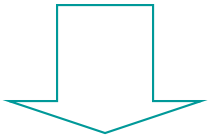
- Assumption:
  - the input is generated by a random process that distributes elements uniformly over [0, 1)
- Idea:
  - Divide [0, 1) into k equal-sized buckets ($k=\Theta(n)$)
  - Distribute the n input values into the buckets
  - Sort each bucket (e.g., using quicksort)
  - Go through the buckets in order, listing elements in each one

- **Input:** $A[1 \ldots n]$, where $0 \le A[i] < 1$ for all i

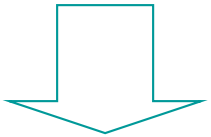
- **Output:** elements $A[i]$ sorted

Example: first pass

12	58	37	64	52	36	99	63	18	9	20	88	47



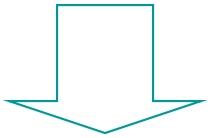
20		12 52	63	64		36	37 47	58 18 88	9 99			



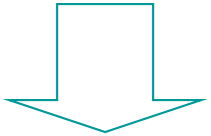
20	12	52	63	64	36	37	47	58	18	88	9	99

Example: second pass

20	12	52	63	64	36	37	47	58	18	88	9	99



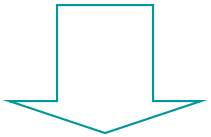
9	12 18	20	36 37	47	52 58	63 64		88	99		



9	12	18	20	36	37	47	52	58	63	64	88	99

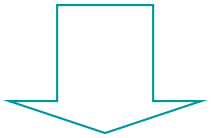
Example: 1st and 2nd passes

12	58	37	64	52	36	99	63	18	9	20	88	47



sort by rightmost digit

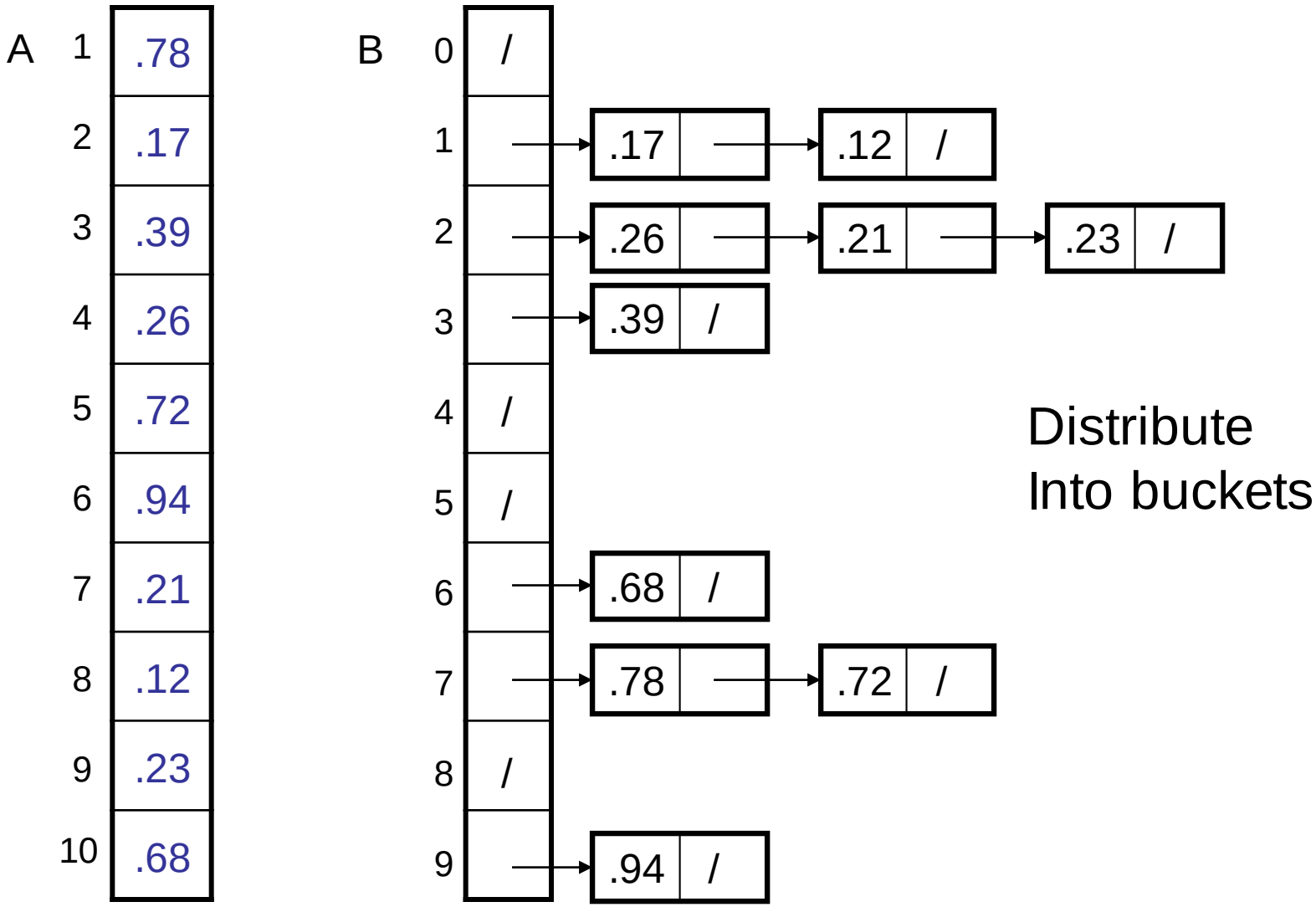
20	12	52	63	64	36	37	47	58	18	88	9	99



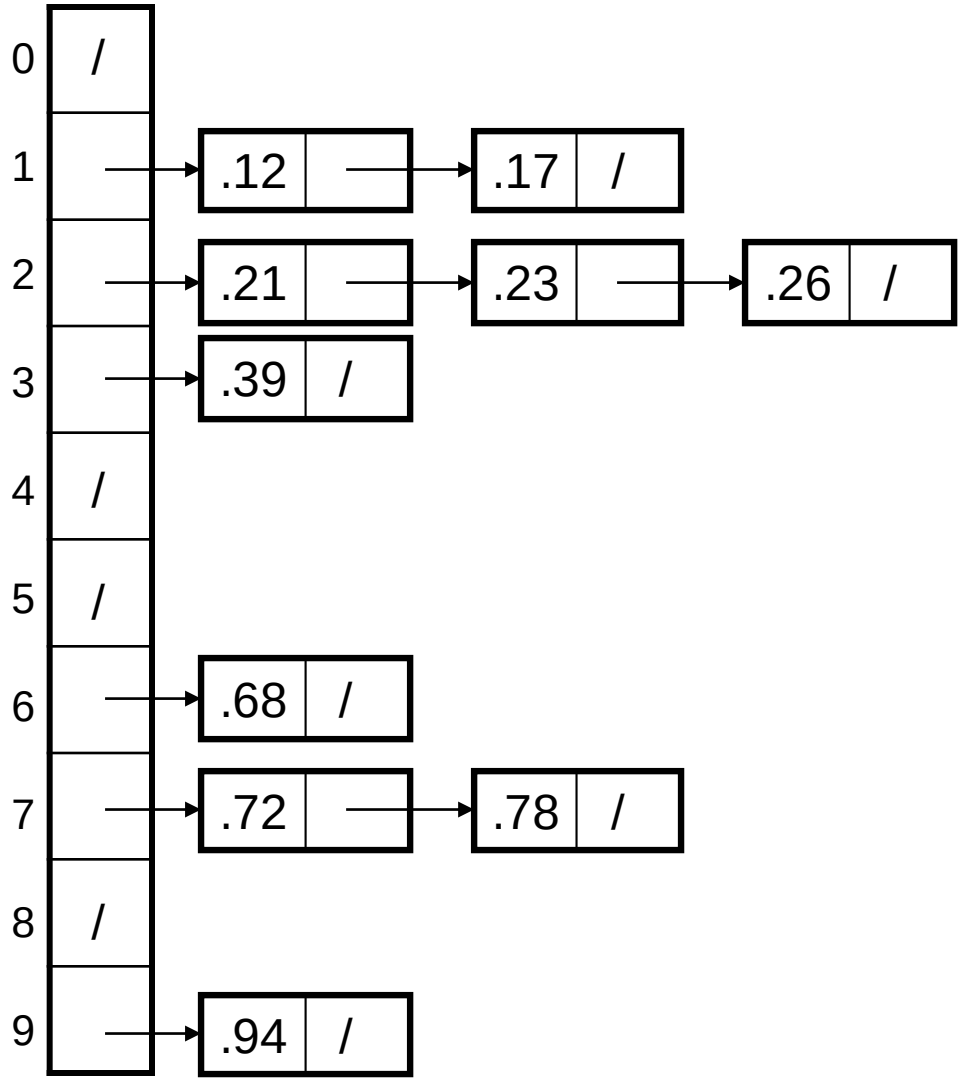
sort by leftmost digit

9	12	18	20	36	37	47	52	58	63	64	88	99

Example - Bucket Sort

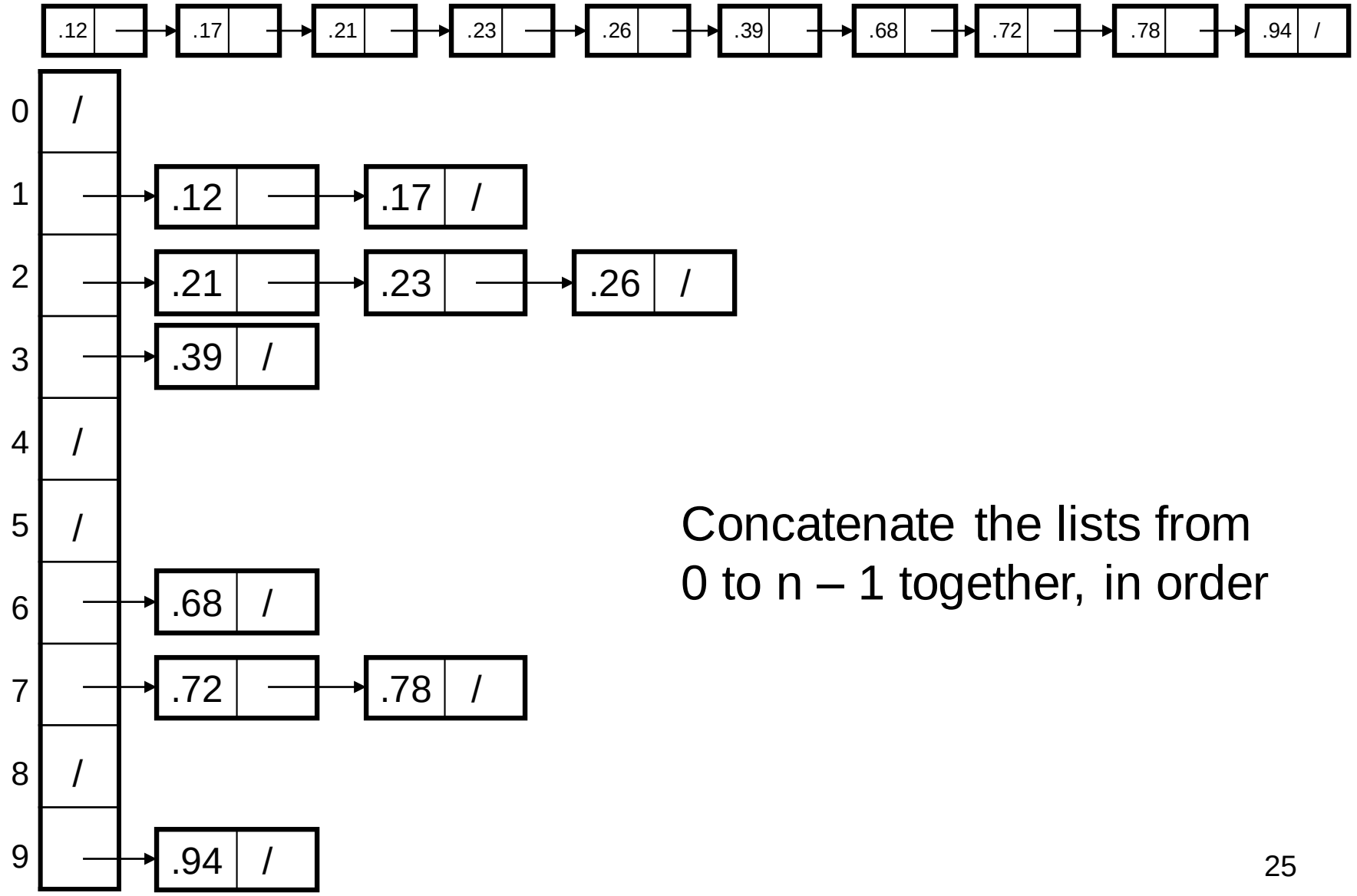


Example - Bucket Sort



Sort within each bucket

Example - Bucket Sort



Concatenate the lists from 0 to n – 1 together, in order

Analysis of Bucket Sort

Alg.: BUCKET-SORT(A, n)

for i ← 1 **to** n

    do insert A[i] into list B[⌊nA[i]⌋]

O(n)

for i ← 0 **to** k - 1

        do sort list B[i] with quicksort sort

k O(n/k log(n/k))
=O(nlog(n/k)

concatenate lists B[0], B[1], . . . , B[n -1]
together in order

return the concatenated lists

O(k)

O(n) (if k=Θ(n))

Summary

- Bucket sort and Radix sort are O(n) algorithms only because we have imposed restrictions on the input list to be sorted
- Sorting, in general, can be done in O(n log n) time